

P Y T H O N D S & A T E A M P R O J E C T

Evaluation Guidance

How your final project will be graded — what to submit, what to demo, what to be ready to answer.

200

T O T A L P O I N T S

What we are evaluating

Not just whether the game runs — whether your data structures and algorithms are used for the right reasons.

01

Working game

From start to end, on the AI+X Studio computer, without complex setup.

02

Justified DS / algorithms

Each of the 6 core features must use a DS/algorithm — and you must defend the choice.

03

Logical explanation

Be able to compare alternatives, discuss complexity, and point to the exact lines of code.

What to submit by Friday

Two deliverables — both must be ready before the demo session.

01

Project PPT

Must include:

- Overall game structure
- Each of the 6 core features
- DS or algorithm used for each feature
- Why that algorithm was chosen
- Comparison with alternative algorithms
- Exact code location where it is used

02

Algorithm explanation

Either a separate document or inline comments in the code.

For every core feature, the writeup or the comments should answer:

- Which DS/algorithm is used
- Why it fits this feature
- Difference vs. alternatives
- Time / space complexity
- Location in the codebase

Six features you must implement

Each one carries 15 points — "it works" alone is not enough. You must defend the DS/algorithm choice.

01

Dungeon Map

2D grid · adjacency · graph rep.

02

Undo System

Stack · LIFO history

03

Turn Management

Queue · or Priority Queue

04

Item Inventory

List · Dict · Tree

05

Enemy AI

BFS · DFS · A pathfinding*

06

Leaderboard

Sort · Heap (top-k)

How the 200 points are distributed

Six categories — implementation and explanation are weighted roughly equally.

Category	Points	Weight
PPT & algorithm submission	30%	
AI+X Studio computer compatibility		
6 core features (15 pts × 6)	15%	
Algorithm comparison & explanation	10%	
interview	25%	
Live game demo	20%	
TOTAL	200 pts	100%

Inside the two biggest categories

Per feature

Feature is actually implemented in the code

DS / algorithm choice is justified

Comparable to alternative algorithms

Code quality & interview

Readable function / class / file structure

Clear variable & function names

Low duplication, maintainable

Handles exceptions properly

Comments / docs where useful

Understands & answers questions

Connects code with algorithm

Re-asks unclear questions; logical answers

Sample questions the evaluator may ask

This is just Example!!

● **Dungeon Map**

Which DS represents the map? Why? How is a movable path checked — compare with BFS / DFS.

● **Turn Management**

Why Queue for turns? If characters have different speeds, is plain Queue enough — vs. Priority Queue.

● **Enemy AI**

How does Enemy AI chase the player? BFS or A* — which fits better here, and why?

● **Undo System**

Why is a Stack a good fit for Undo? Do you store full state or only deltas — trade-offs?

● **Item Inventory**

List vs. Dict vs. Tree for the inventory? When does Tree win for search and sort?

● **Leaderboard**

How are scores sorted? If only top-10 matters, do you really need a full sort — compare Heap.

A 5-step answer beats a 1-step answer

Practiced teams answer in this order. Doing all five gets you the full "interview" points.

1

State the DS / algorithm

"We used a Stack." One sentence — no ramble.

2

Justify the choice

"Undo is LIFO, so a Stack matches the access pattern."

3

Point at the code

"It lives in player.py, in the undo() method."

4

Compare an alternative

"A Queue would be FIFO and would replay actions in the wrong order."

5

Mention complexity / limits

"Push and pop are $O(1)$; memory grows with history length."

Worked example

"Undo uses a Stack. Undo is LIFO, so it matches Stack's access pattern. It's in player.undo() — we push the state before each move and pop on undo. Queue would be FIFO and replay actions in the wrong order. Push / pop are $O(1)$; memory grows with history length."

Be ready to compare these by hand

This is for Example!!!

Linear DS	Non-linear DS	Search / Pathfinding	Sorting & ranking
- List - Stack (LIFO) - Queue (FIFO) - Deque	- Tree - Binary Search Tree - Heap (Priority Queue) - Graph	- BFS — shortest in unweighted - DFS — exhaustive - A* — heuristic-guided - Dijkstra — weighted shortest	- Sorted full list — $O(n \log n)$ - Heap top-k — $O(n \log k)$ - Insertion / Bubble — $O(n^2)$ - Hash lookup — $O(1)$ avg.

Before you submit — work through this list

Please check your check-lists

- | | |
|--|--|
| ☐ We ran the game on the AI+X Studio computer (not just our laptops). | ☐ We can state the time or space complexity of each. |
| ☐ The game runs from start to game-over without crashes. | ☐ The PPT matches what the code actually does. |
| ☐ All 6 core features are present in the code. | ☐ We know the file and function name where each feature is implemented. |
| ☐ We can name the DS / algorithm used for each feature. | ☐ Each team member has practiced answers to the 12 sample questions. |
| ☐ We can compare each algorithm with at least one alternative. | |

THE THREE THINGS THAT MATTER

Run it. Defend it. Compare it.

01

Run it

Game starts → plays → ends, on the AI+X Studio computer, every time.

02

Defend it

For every DS and algorithm choice, name the reason out loud — not just "it worked".

03

Compare it

Know at least one alternative for each feature and the trade-off in time or space.

Good luck — show us not just what your game does, but why you built it that way.